App Summary

Repo: my-nextjs-app

What It Is

A Next.js 16 web app that bundles browser-based utilities with a few server-backed workflows, including AI Q&A, email sending, user auth, and a real-time chat room.

The repo reads like an internal toolbox or personal utility hub rather than a single-purpose product.

Who It Is For

Primary persona: developers or technical operators who need a lightweight web hub for data/text utilities, quick internal workflows, and a protected AI knowledge base. Persona is inferred from the route set and admin/auth features.

What It Does

- Provides utility pages for JSON formatting, Base64, text diff, time conversion, data filtering, EML viewing, Brainfuck, and calculators.
- Supports AI Q&A over uploaded TXT, MD, and PDF files using chunking, embeddings, retrieval, and an LLM-backed answer route.
- Offers login and registration flows with email verification, captcha, JWT cookie sessions, and admin-only user management.
- Includes a token-based real-time chat room backed by persisted messages plus Server-Sent Events for live updates.
- Sends email through Gmail SMTP with validation, attachment handling, and in-memory rate limiting.
- Runs a daily mail cron endpoint that appends a BTC snapshot and chart when configured.

How It Works

App Router pages under app/* render the UI and call route handlers under app/api/*. Middleware protects /ai-qa and /api/ai-qa by validating a JWT session cookie. Route handlers use Prisma to read/write Postgres tables for users, verification tokens, chat messages, and document chunks.

AI ingest flow: upload files -> parse PDF or text -> split into chunks -> generate embeddings with Google or OpenAI -> store vectors in DocumentChunk. AI chat flow: embed query -> score stored chunks with cosine similarity -> build context prompt -> call Gemini or GPT model. Chat flow: POST message -> save via Prisma -> notify in-process SSE bus -> /api/chat/stream pushes history and live events. Mail flow: UI or cron route -> validation and rate limit -> Nodemailer Gmail transport.

How To Run

- Create a .env with at least PRISMA_DATABASE_URL and a 32+ character JWT_SECRET. Add GMAIL_USER/GMAIL_APP_PASSWORD and GOOGLE_API_KEY or OPENAI_API_KEY only if using mail or AI features.
- Install dependencies: npm install
- Initialize Prisma for the local database: npm run prisma:generate and npm run prisma:migrate
- Start the app: npm run dev, then open http://localhost:3000

Not found in repo: a single canonical env example file or seed script.